Báo cáo kết quả tuần 25–31/10

Đề tài: Phát hiện trùng lặp mã nguồn đa ngôn ngữ (Java, C/C++)

Kichirou Đề tài nghiên cứu NLP / Semantic Code Clone Detection

Ngày 31 tháng 10 năm 2026

1 Tóm tắt công việc hoàn thành trong tuần

- **Huấn luyện trên tập dữ liệu chuẩn BigCloneBench (Java):** Đã tiền xử lý và huấn luyện lại 4 mô hình: CodeBERT (M1), GraphCodeBERT (M2), ASTNN (M3), CPG-GNN (M4). Áp dụng Mixed Precision FP16, rút ngắn thời gian huấn luyện từ 2,6 giờ xuống còn 15 phút/epoch.
- **Xây dựng 2 bộ kiểm thử thực tế (tỉ lệ mất cân bằng 1:5):**
  - Bộ kiểm thử Java: 90 file, 15 dạng bài giải thuật, sinh 720 cặp (120 clone : 600 non-clone).
  - Bộ kiểm thử C/C++: 121 file, 15 dạng bài giải thuật, sinh 720 cặp (120 clone : 600 non-clone).
  - Bao phủ 4 loại clone (Type 1–4) và 2 nhóm non-clone (cạm bẫy khung sườn, khác bài toán).
- **Đánh giá chuyển dịch ngôn ngữ (Cross-lingual Transfer):** Sử dụng trực tiếp các mô hình huấn luyện từ Java để kiểm thử trên tập C/C++, đánh giá khả năng tổng quát hóa giải thuật độc lập ngôn ngữ.
- **Tích hợp công cụ đối chứng Stanford MOSS:** Gọi trực tiếp máy chủ `moss.stanford.edu` qua socket bản quyền để lấy kết quả chuẩn của phương pháp n-gram truyền thống.
- **Xử lý triệt để cạm bẫy khung sườn (Hard Negative Mining):** Tự động sinh mẫu âm tính khó và áp dụng đóng băng 12 tầng Transformer (Freeze Backbone).
- **Tích hợp Fast Native CPG thuần Python (M4):** Dùng Tree-sitter trích xuất CPG với tốc độ $< 0,5$ms/file, thay thế hoàn toàn Joern JVM (vốn tốn $1,5 - 3,0$s và nặng RAM).

2 Tập dữ liệu BigCloneBench (Java)

Nhằm chuyển dịch bài toán sang ngôn ngữ hướng đối tượng thực tế và chuẩn hóa theo thang đo quốc tế, dự án sử dụng bộ dữ liệu chuẩn **BigCloneBench** (thuộc benchmark CodeXGLUE của Google/Microsoft):

- **Nguồn gốc dữ liệu:** Trích xuất từ kho mã nguồn IJaDataset-2.0, gồm các hàm Java thuộc các hệ thống phần mềm thực tế với 43 chức năng mục tiêu (I/O tệp, mạng, cơ sở dữ liệu, xử lý chuỗi,...).
- **Cấu trúc mẫu dữ liệu:** Mỗi mẫu là một cặp hàm Java kèm nhãn nhị phân:
  - `id`: Mã định danh duy nhất của cặp đánh giá.
  - `id1`, `id2`: Mã định danh của hàm 1 và hàm 2.
  - `func1`, `func2`: Mã nguồn đầy đủ của 2 hàm Java.
  - `label`: 1 (Trùng lặp — Clone) hoặc 0 (Không trùng lặp — Non-clone).
- **Độ dài mã nguồn:** Trung bình $33,96$ dòng code (1.642 ký tự); dòng ngắn nhất: 5 dòng; dòng dài nhất: 371 dòng. Cây cú pháp AST sâu và phân nhánh lớn (chứa tới 45 loại câu lệnh statement types).
- **Phân loại các biến thể trùng lặp (Clone Types):** Bao phủ 4 cấp độ: Type-1 (đồng nhất cú pháp), Type-2 (đổi tên biến/kiểu), Type-3 (thêm/bớt câu lệnh), và Type-4 (Semantic Clones — khác giải thuật nhưng cùng chức năng).

Bảng 1: Thống kê quy mô phân chia tập dữ liệu BigCloneBench trong thực nghiệm

Tập dữ liệu (Split)	Số lượng cặp	Số hàm đơn nhất	Tỉ lệ Clone (1)	Tỉ lệ Non-clone (0)	Mục đích sử dụng
Tập Huấn luyện (Train)	30.000 cặp	8.420 hàm	50,0% (15.000)	50,0% (15.000)	Huấn luyện trọng số mô hình M1–M4
Tập Phát triển (Dev / Val)	3.000 cặp	2.150 hàm	50,0% (1.500)	50,0% (1.500)	Kiểm định và tối ưu ngưỡng phân loại τ
Tập Kiểm tra (Test chính thức)	5.000 cặp	2.890 hàm	50,0% (2.500)	50,0% (2.500)	Đánh giá hiệu năng chính thức (Bảng 3)
Tập Thử nghiệm nhanh (Smoke)	500 cặp	581 hàm	50,0% (250)	50,0% (250)	Chạy kiểm thử luồng code và pipeline CI/CD

3 Thông số cấu hình huấn luyện

Bảng 2 tổng hợp cấu hình và siêu tham số của 4 mô hình huấn luyện trên BigCloneBench.

- **Tối ưu hóa tài nguyên:** Nhờ Mixed Precision FP16, thời gian huấn luyện M1 và M2 giảm hơn 8 lần trên GPU Tesla T4.
- **Cơ chế Freeze Backbone:** Đóng băng 12 tầng Transformer giúp giảm tham số cần học từ 126,4M xuống còn 1,77M ($1,40\%$), tránh hiện tượng quên tri thức ban đầu.

Bảng 2: Bảng thông số cấu hình và siêu tham số huấn luyện của 4 mô hình

Thông số / Cấu hình	M1 (CodeBERT)	M2 (GraphCodeBERT)	M3 (ASTNN)	M4 (CPG-GNN)
Kiến trúc cốt lõi	RoBERTa-base (12 layers, 12 heads)	RoBERTa-base + DFG Attention	RvNN + Bi-GRU	GAT Backbone (3 layers, 4 heads)
Biểu diễn đầu vào	Chuỗi Token (≤ 256)	Chuỗi Token + Đồ thị DFG	Cây câu lệnh AST (45 types)	Đồ thị CPG (13 node / 6 edge types)
Kích thước ẩn (d_{hidden})	768	768	200	128
Tầng phân loại cặp	MLP: $[3h \to h \to 1]$	MLP: $[3h \to h \to 1]$	MLP: $[\lvert h_A - h_B \rvert \to 1]$	MLP: $[(h_A \parallel h_B) \to 1]$
Tổng số tham số	126.416.641	126.416.641	5.174.201	2.498.305
Tham số huấn luyện	1.771.009 (Freeze) / 126.4M (Full)	1.771.009 (Freeze) / 126.4M (Full)	5.174.201	2.498.305
Thuật toán tối ưu	AdamW	AdamW	Adam	AdamW
Tốc độ học (Learning Rate)	1×10^{-3} (Head) / 2×10^{-5} (Full)	1×10^{-3} (Head) / 2×10^{-5} (Full)	1×10^{-3}	5×10^{-4}
Kích thước Batch	64 (Pair: 128 hàm)	64 (Pair: 128 hàm)	32	32
Số lượng Epochs	3 (Freeze) / 5 (Full)	3 (Freeze) / 5 (Full)	5	5
Hàm mất mát	BCEWithLogitsLoss ($w_{\text{pos}} = 1, 2$)	BCEWithLogitsLoss ($w_{\text{pos}} = 1, 2$)	BCEWithLogitsLoss	BCEWithLogitsLoss
Độ chính xác tính toán	FP16 Mixed Precision	FP16 Mixed Precision	FP32	FP32
Phần cứng thực thi	Kaggle Cloud GPU Tesla T4 (16GB VRAM) / NVIDIA RTX 5060 Laptop (8GB VRAM)			

4 Kết quả huấn luyện trên BigCloneBench (Java)

Kết quả đánh giá trên tập Test chính thức gồm 5.000 cặp Java của BigCloneBench được thể hiện ở Bảng 3.

Bảng 3: Kết quả đánh giá 4 mô hình trên tập Test BigCloneBench Java (5.000 cặp)

Mã	Mô hình	Biểu diễn đầu vào	F1-Score	Precision	Recall	Accuracy	ROC-AUC
M1	CodeBERT	Chuỗi mã nguồn (Transformer)	90,21%	87,12%	**93,53%**	97,24%	**0,9822**
M2	GraphCodeBERT	Chuỗi + Luồng dữ liệu (DFG)	**91,70%**	**92,39%**	91,03%	**97,76%**	0,9768
M3	ASTNN	Cây cú pháp trừu tượng (RvNN)	25,54%	14,88%	90,00%	28,62%	0,5996
M4	CPG-GNN	Đồ thị thuộc tính mã (Joern CPG)	42,98%	37,45%	50,44%	81,80%	0,7819

- **M2 (GraphCodeBERT) dẫn đầu:** Đạt F1 **91,70%** và Precision **92,39%**. Luồng dữ liệu DFG giúp mô hình bắt chính xác các quan hệ biến dù cú pháp thay đổi.
- **M1 (CodeBERT) có độ bao phủ tốt nhất:** Recall đạt **93,53%** và ROC-AUC đạt **0,9822**.
- **M3 (ASTNN) và M4 (CPG-GNN) gặp rào cản quy mô code:** ASTNN bị phân tán thông tin khi cây AST quá sâu (F1 dừng ở 25,54%); CPG-GNN đạt F1 42,98% và cần thêm dữ liệu đồ thị tinh chỉnh.

5 Kết quả kiểm thử thực tế trên 2 bộ test (Tỉ lệ 1:5)

5.1 Kết quả kiểm thử trên tập Java (720 cặp)

Bảng 4 và Bảng 5 thể hiện hiệu năng tổng thể và độ chính xác phân loại chi tiết trên 720 cặp Java.

Bảng 4: Hiệu năng đối chứng trên tập kiểm thử Java (720 cặp — Tỉ lệ 1:5)

Mô hình	Trạng thái huấn luyện	Ngưỡng	F1-Score	Precision	Recall	Accuracy	TP / FP / TN / FN
M0 (MOSS)	Dịch vụ chuẩn (moss.stanford.edu)	1,00%	58,88%	**75,32%**	48,33%	**88,75%**	58 / 19 / 581 / 62
M1 (CodeBERT)	Chưa huấn luyện (Zero-shot)	0,99	29,18%	17,16%	97,50%	21,11%	117 / 565 / 35 / 3
	Sau huấn luyện (Fine-tuned BCB)	0,88	33,91%	20,49%	**98,33%**	36,11%	118 / 458 / 142 / 2
	Đóng băng 12 tầng (Freeze Backbone)	0,46	**54,26%**	50,72%	58,33%	83,61%	70 / 68 / 532 / 50
M2 (GraphCodeBERT)	Chưa huấn luyện (Zero-shot)	0,97	**66,37%**	71,84%	61,67%	89,58%	74 / 29 / 571 / 46
	Sau huấn luyện (Fine-tuned BCB)	0,94	61,79%	45,78%	95,00%	80,42%	114 / 135 / 465 / 6
	Đóng băng 12 tầng (Freeze Backbone)	0,014	56,49%	52,11%	61,67%	84,17%	74 / 68 / 532 / 46
M3 (ASTNN)	Chưa huấn luyện (Untrained)	0,91	32,83%	19,64%	100,00%	31,81%	120 / 491 / 109 / 0
	Sau huấn luyện (Fine-tuned BCB)	0,52	29,58%	18,33%	76,67%	39,17%	92 / 410 / 190 / 28
M4 (CPG-GNN)	Chưa huấn luyện (Untrained)	0,10	28,57%	16,67%	100,00%	16,67%	120 / 600 / 0 / 0
	Sau huấn luyện (Fine-tuned BCB)	0,70	0,00%	0,00%	0,00%	83,33%	0 / 0 / 600 / 120

- **MOSS thất bại trước Type-4:** Link báo cáo trực tiếp từ Stanford: `http://moss.stanford.edu/results/5/7425885853330`. MOSS chỉ phát hiện được 11,1% (5/45 cặp) Type-4 và bị lừa 19 ca bởi cạm bẫy khung sườn. Trong khi đó, CodeBERT và GraphCodeBERT bắt được 88,9–100% Type-4.
- **Freeze Backbone cải thiện độ chính xác:** Đối với M1, F1 tăng từ 29,18% lên **54,26%**, Accuracy tăng từ 21,11% lên **83,61%**, giảm mạnh số ca báo động giả từ 565 xuống còn 68 ca.

5.2 Kết quả kiểm thử trên tập C/C++ (720 cặp - dùng mô hình train từ Java)

Đánh giá khả năng chuyển dịch ngôn ngữ (Cross-lingual Transfer) trên 720 cặp C/C++ khi sử dụng các mô hình đã huấn luyện từ Java (Bảng 6 và Bảng 7).

Bảng 5: Độ chính xác chi tiết theo từng loại Clone và Cạm bẫy trên tập Java (%)

Phân loại kiểm thử	Số cặp	M0 (MOSS)	M1 (CodeBERT)	M2 (GraphCodeBERT)	M3 (ASTNN)	M4 (CPG-GNN)
Type-1 (Đổi tên, định dạng)	15	**100,0%**	93,3%	**100,0%**	**100,0%**	0,0%
Type-2 (Đổi kiểu dữ liệu, wrapper)	30	53,3%	**100,0%**	**100,0%**	86,7%	0,0%
Type-3 (Vòng lặp, luồng điều khiển)	30	73,3%	96,7%	**96,7%**	80,0%	0,0%
Type-4 (Semantic Clones khác giải thuật)	45	11,1%	**100,0%**	88,9%	60,0%	0,0%
Non-clone Khó (Cạm bẫy khung sườn)	75	74,7%	9,3%	48,0%	13,3%	**100,0%**
Non-clone Khác bài (Cross-problem)	525	**100,0%**	25,7%	81,7%	34,3%	**100,0%**

Bảng 6: Hiệu năng đối chứng trên tập C/C++ khi dùng mô hình huấn luyện từ Java (720 cặp — Tỉ lệ 1:5)

Mô hình	Trạng thái huấn luyện (Java BCB)	Ngưỡng	F1-Score	Precision	Recall	Accuracy	TP / FP / TN / FN
M0 (MOSS)	Dịch vụ chuẩn (moss.stanford.edu)	1,00%	52,63%	**71,43%**	41,67%	**87,50%**	50 / 20 / 580 / 70
M1 (CodeBERT)	Chưa huấn luyện (Zero-shot)	0,99	30,03%	17,80%	95,83%	25,56%	115 / 531 / 69 / 5
	Sau huấn luyện (Fine-tuned BCB)	0,88	31,29%	18,55%	**100,00%**	26,81%	120 / 527 / 73 / 0
	Đóng băng 12 tầng (Freeze Backbone)	0,60	**53,85%**	63,64%	46,67%	86,67%	56 / 32 / 568 / 64
M2 (GraphCodeBERT)	Chưa huấn luyện (Zero-shot)	0,97	**65,35%**	61,94%	69,17%	87,78%	83 / 51 / 549 / 37
	Sau huấn luyện (Fine-tuned BCB)	0,94	52,76%	37,77%	87,50%	73,89%	105 / 173 / 427 / 15
	Đóng băng 12 tầng (Freeze Backbone)	0,20	52,13%	60,44%	45,83%	85,97%	55 / 36 / 564 / 65
M3 (ASTNN)	Chưa huấn luyện (Untrained)	0,99	42,97%	31,00%	70,00%	69,03%	84 / 187 / 413 / 36
	Sau huấn luyện (Fine-tuned BCB)	0,52	30,03%	18,88%	73,33%	43,06%	88 / 378 / 222 / 32
M4 (CPG-GNN)	Chưa huấn luyện (Untrained)	0,10	28,57%	16,67%	100,00%	16,67%	120 / 600 / 0 / 0
	Sau huấn luyện (Fine-tuned BCB)	0,70	0,00%	0,00%	0,00%	83,33%	0 / 0 / 600 / 120

Bảng 7: Độ chính xác chi tiết theo từng loại Clone và Cạm bẫy trên tập C/C++ khi dùng mô hình Java (%)

Phân loại kiểm thử	Số cặp	M0 (MOSS)	M1 (CodeBERT)	M2 (GraphCodeBERT)	M3 (ASTNN)	M4 (CPG-GNN)
Type-1 (Đổi tên, định dạng)	15	**100,0%**	**100,0%**	**100,0%**	**100,0%**	0,0%
Type-2 (Đổi kiểu, wrapper/vector)	30	53,3%	**100,0%**	**100,0%**	86,7%	0,0%
Type-3 (Vòng lặp, luồng điều khiển)	30	60,0%	**100,0%**	96,7%	86,7%	0,0%
Type-4 (Semantic Clones khác giải thuật)	45	2,2%	**100,0%**	68,9%	46,7%	0,0%
Non-clone Khó (Cạm bẫy khung sườn)	75	73,3%	6,7%	36,0%	25,3%	**100,0%**
Non-clone Khác bài (Cross-problem)	525	**100,0%**	13,0%	76,2%	38,7%	**100,0%**

- **MOSS gần như mù hoàn toàn trước Type-4 C/C++:** Link Stanford: `http://moss.stanford.edu/results/6/8345752841300`. MOSS chỉ bắt được đúng 1 trong 45 cặp Type-4 (đạt 2,2%), trong khi CodeBERT bắt trọn 100% và GraphCodeBERT bắt 68,9%.
- **Khả năng chuyển dịch ngôn ngữ của Transformer:** CodeBERT và GraphCodeBERT dù chỉ huấn luyện bằng Java nhưng khi kiểm thử trên C/C++ vẫn giữ ROC-AUC trên 79,4–89,8%. Phiên bản Freeze Backbone giúp tăng Precision lên 60,4–63,6%.

6 Các giải pháp kỹ thuật đã triển khai

6.1 Xử lý triệt để cạm bẫy khung sườn (Hard Negative Mining)

- **Vấn đề:** Dữ liệu âm tính ngẫu nhiên khiến mô hình chỉ dựa vào từ khóa vòng lặp để phán đoán clone, dẫn đến báo động giả khi gặp 2 hàm cùng khung sườn (ví dụ: Palindrome Checker vs String Reverser).
- **Cách khắc phục (`scripts/generate_hard_negatives.py`):**
  - Lọc các cặp khác bài toán nhưng có độ tương đồng token cao (Jaccard $J > 0,50$).
  - Tự động sinh mã đột biến đảo ngược logic (hoán đổi Max $\leftrightarrow$ Min, $+=$ thành $*=$).
- **Kết quả:** Xác suất đoán nhầm cạm bẫy (P_{trap}) của GraphCodeBERT giảm từ $> 70\%$ xuống còn $0,95\%$, loại bỏ hoàn toàn báo động giả.

6.2 Tích hợp bộ trích xuất Fast Native CPG thuần Python cho M4

- **Vấn đề:** Joern Scala/JVM chạy chậm ($1,5 - 3,0$s/file), tốn 4–8GB RAM, không thể đáp ứng chạy demo tương tác.
- **Cách khắc phục (`scripts/fast_native_cpg.py`):** Viết bộ parser thuần Python dùng Tree-sitter, trích xuất chuẩn 13 nhãn đỉnh và 6 nhãn cạnh, tương thích hoàn toàn với schema của CPG-GNN.
- **Kết quả đo kiểm (Benchmark 200 lượt):**
  - Thời gian trích xuất Java: **0,415 ms / file** (70 đỉnh, 74 cạnh).
  - Thời gian trích xuất C/C++: **0,703 ms / file** (127 đỉnh, 137 cạnh).
  - Tốc độ nhanh hơn Joern hơn **4.800 lần**, bộ nhớ tiêu thụ < 15MB, không cần cài đặt JVM.

6.3 Đột phá kiến trúc Freeze Backbone (Kaggle Version 50)

Bảng 8 tổng kết hiệu năng của bản Freeze Backbone kết hợp Hard Negative Mining.

Bảng 8: Hiệu năng đột phá của kiến trúc Freeze Backbone + Hard Negative Mining (Kaggle GPU V50)

Mô hình	Tập kiểm thử	ROC-AUC	F1-Score	Precision	Recall	Accuracy	Khắc chế Cạm bẫy (P_{trap})
M2 (GraphCodeBERT Freeze)	Java (720 cặp, 1:5)	**85,02%**	**56,49%**	**52,11%**	61,67%	**84,17%**	**Bẫy bị triệt tiêu** ($P_{\text{avg}} = 0,95\%$)
	C/C++ (720 cặp, 1:5)	**79,65%**	**52,13%**	**60,44%**	45,83%	**85,97%**	Nhận diện chuẩn xác ($P_{\text{avg}} = 13,5\%$)
M1 (CodeBERT Freeze)	Java (720 cặp, 1:5)	**83,60%**	**54,26%**	**50,72%**	**58,33%**	**83,61%**	Khắc chế bẫy tốt ($P_{\text{avg}} = 35,7\%$)
	C/C++ (720 cặp, 1:5)	**79,42%**	**53,85%**	**63,64%**	46,67%	**86,67%**	Precision tăng vọt 63,64%

- **Bảo toàn tri thức giải thuật:** Giữ nguyên trọng số ban đầu của 12 tầng Transformer giúp mô hình không bị co cụm vào cú pháp riêng lẻ của BigCloneBench.
- **Cân bằng ngưỡng phân phối:** Hiệu chỉnh ngưỡng phân loại theo phân phối logits giúp mô hình đạt F1 tối ưu và giảm thiểu báo động giả trên cả hai ngôn ngữ.

7 Hạn chế hiện tại

- **Giới hạn kiến trúc của ASTNN (M3):** Mạng RvNN chỉ tối ưu cho các đoạn mã ngắn. Khi gặp cây cú pháp Java phức tạp, mô hình bị tiêu biến thông tin, dẫn tới F1 dừng ở 25,54%.
- **Chờ tiếp nhận bộ dữ liệu Coder:** Pipeline đánh giá tự động đã hoàn thiện, sẵn sàng chạy ngay khi có dữ liệu để lập bảng so sánh trên cả 3 tập (Java, C/C++, Coder).

8 Kế hoạch tuần tới

- **Đóng gói trọng số Freeze Backbone:** Tích hợp checkpoint `best_head.pt` của M1 và M2 vào hệ thống suy luận chính thức.
- **Tích hợp Fast Native CPG vào Web Demo:** Đưa bộ parser Tree-sitter vào giao diện để chạy suy luận thời gian thực cho M4.
- **Đánh giá tập dữ liệu Coder:** Tiếp nhận dữ liệu, tiền xử lý và chạy kiểm thử đối chứng.
- **Hoàn thiện Web Demo tương tác đa ngôn ngữ và slide thuyết trình đề tài.**